

Kod Geometrisinin Uygulamalı Analizi: Bir Kod Tabanı Üzerinde Niceliksel Vaka İncelemesi

Ömer Yavuz

May 2026

Giriş

Kod tabanı geometrisi yaklaşımı, bir yazılım sistemini modüllerin düğüm, içe aktarma ilişkilerinin kenar, dosya hacminin kütle ve içe aktarma yoğunluğunun merkezî çekim olarak yorumlandığı geometrik bir yapı biçiminde ele almaktadır. Bu çalışmada söz konusu yaklaşım belirli bir kod tabanı üzerinde uygulanmakta; soyut mimari kavramların gerçek ölçümlerle nasıl desteklenebileceği gösterilmektedir. Aşağıdaki sayılar yalnızca incelenen kod tabanına özgüdür ve doğrudan başka projelere taşınmaz; ancak yöntem evrenseldir. İncelenen sistemdeki sayfa ve dosya adları anonimleştirilerek sunulmuştur.

Kavramsal Arka Plan

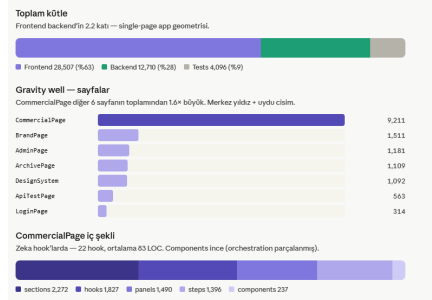
Analizde üç temel kavram kullanılmaktadır. **Kütle**, bir modülün kod tabanı içindeki hacmini (LOC) ifade etmektedir. **Merkezîlik**, bir dosyanın diğer modüller tarafından ne ölçüde içe aktarıldığını göstermektedir. **Gravity well**, sistemde olağan dışı miktarda sorumluluk, bağımlılık veya invariant çeken; sistem davranışını merkezileştiren ve yeniden yapılandırma riskini artıran modüldür.

Ölçüm Metodolojisi

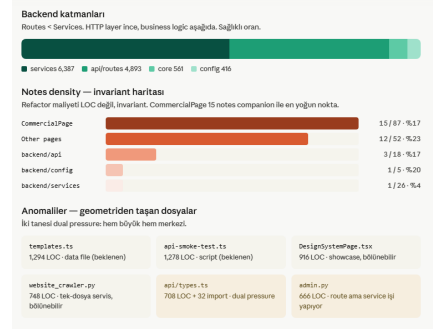
Analizde kullanılan niceliksel veriler üç ana ölçüm üzerinden elde edilmiştir:

1. **LOC ölçümü:** Modül, klasör ve dosya bazında kod satırı yoğunluğu hesaplanmıştır.
2. **İçe aktarma merkezîliği:** En çok içe aktarılan dosyalar tespit edilerek yapısal çekim noktaları belirlenmiştir.
3. **Notes companion yoğunluğu:** Mimari invariant taşıyan not dosyalarının hangi bölgelerde yoğunlaştığı incelenmiştir.

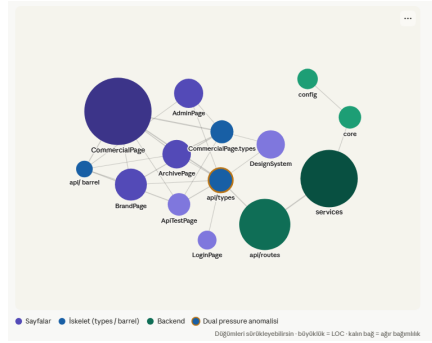
Bu ölçümler tek başına mimari tanı üretmez; tanı, sayısal ölçümlerin yönlü grafik modeliyle birlikte yorumlanmasıyla elde edilmektedir.



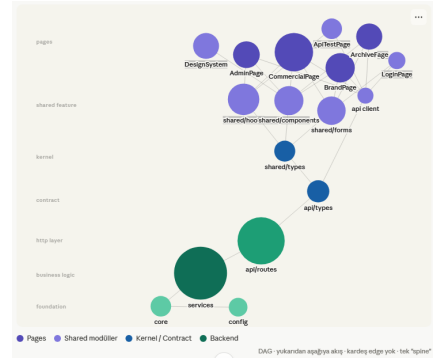
(a) Kütle ve dağılım görselleştirilmesi.



(b) Sayfa bazlı yoğunluk.



(c) Merkezilik ve bağımlılık ağı.



(d) Notes companion yoğunluk haritası.

Figure 1: Kod tabanının görsel geometrik teşhisi.

Toplam Kütle Dağılımı

Katman	LOC	Oran
Frontend	28,507	%63
Backend	12,710	%28
Tests	4,096	%9

Table 1: Kod tabanının toplam kütle dağılımı.

Frontend tarafı backend'e göre yaklaşık 2.2 kat daha büyüktür. Bu durum tek başına patolojik değildir; single-page application yapılarında kullanıcı yüzeyi doğal olarak daha hacimli olabilmektedir. Belirleyici soru, bu frontend kütlelerinin sayfalar arasında nasıl dağıldığıdır.

Sayfa	LOC
Page A (ana işlem sayfası)	9,211
Page B	1,511
Page C (yönetim)	1,181
Page D (arşiv)	1,109
Page E (tasarım sistemi)	1,092
Page F (API test)	563
Page G (oturum açma)	314

Table 2: Frontend sayfaları arasındaki kütle dağılımı.

Sayfa Bazlı Kütle: Page A Gravity Well

Page A, frontend page katmanının yaklaşık %61'ini taşımaktadır. Diğer altı sayfanın toplamı 5,770 LOC iken, Page A tek başına 9,211 LOC değerindedir. Bu, Page A'nın yalnızca büyük bir sayfa olmadığını; aynı zamanda sistemin merkezi davranış alanlarından biri haline geldiğini göstermektedir.

Page A İç Kütle Dağılımı

Alt Bölge	LOC
sections	2,272
hooks	1,827
panels	1,490
steps	1,396
components	237

Table 3: Page A içerisindeki kütle dağılımı.

Davranışsal karmaşıklık component katmanında değil; hook ve section orkestrasyon yapılarında yoğunlaşmıştır. Components klasörünün görece küçük kalması, görsel parçaların hafif olduğunu; buna karşılık state, orkestrasyon ve akış yönetiminin hook'larda biriktiğini göstermektedir. Bu nedenle Page A için yapılacak yeniden yapılandırma yalnızca bileşen parçalama düzeyinde kalmamalı; hook sınırlarının davranış alanlarına göre yeniden değerlendirilmesini de kapsamalıdır.

Backend Katman Dağılımı

Services katmanının routes katmanından daha büyük olması, iş mantığının büyük ölçüde HTTP katmanının altına indirildiğini göstermektedir; bu, sağlıklı bir katmanlı mimarinin işaretidir. Bununla birlikte bazı route dosyalarının olağan dışı büyümesi, belirli giriş noktalarında orkestrasyon birikimi olabileceğini düşündürmektedir.

Backend Katmanı	LOC
services	6,387
api/routes	4,893
core	561
config	416

Table 4: Backend katmanlarının kütle dağılımı.

Merkezîlik ve İskelet Düzgümler

İçe Aktarma Hedefi	İçe Aktarma Sayısı
../types	60
../PageA.types	40
../api	32
../api	29
../constants	17

Table 5: En çok içe aktarılan frontend dosyaları.

Sistem belirli tip ve API düğümleri etrafında dönmektedir. Özellikle `PageA.types.ts` ve `api/types.ts` dosyaları hem davranışsal hem yapısal açıdan merkezi hale gelmiştir. Yüksek içe aktarma sayısı yanlış ad alanı veya aşırı dosya hacmiyle birleştiğinde mimari baskı üretmekte; bu nedenle bu düğümler yeniden yapılandırma sıralamasında öncelik taşımaktadır.

Notes Companion Yoğunluğu

Bölge	Notes	Dosya	Yoğunluk
Page A	15	87	%17
Diğer sayfalar	12	52	%23
backend/services	1	26	%4
backend/api	3	18	%17
backend/config	1	5	%20

Table 6: Notes companion yoğunluğunun modüller arası dağılımı.

Page A çevresindeki 15 notes companion dosyası, bu bölgenin yalnızca büyük değil; aynı zamanda invariant yoğun bir alan olduğunu göstermektedir. Yeniden yapılandırma maliyeti yalnızca 9K LOC değildir; asıl maliyet bu satırlarda saklanan niyet ve davranış kurallarının doğru taşınmasıdır.

Geometriden Taşan Dosyalar

Dosya	LOC	Yorum
<code>templates.ts</code>	1,294	Veri dosyası; büyüklük beklenebilir.
<code>api-smoke-test.ts</code>	1,278	Test betiği; bağlama göre değerlendirilmeli.
<code>PageE.tsx</code>	916	Showcase niteliğinde; bölünebilir.
<code>crawler.py</code>	748	Tek dosyada yoğunlaşmış backend service adayı.
<code>api/types.ts</code>	708	Boundary contract; domain bazlı bölünme adayı.
<code>admin.py</code>	666	Route düzeyinde orkestrasyon birikimi.

Table 7: Kod tabanındaki olağan dışı büyük veya merkezî dosyalar.

Refactor adayları yalnızca büyüklüklerine göre değil, mimari rollerine göre okunmalıdır. `templates.ts` büyük olabilir; fakat veri dosyası olduğu için yapısal problem üretmemektedir. Buna karşılık `api/types.ts`, hem büyük hem merkezî olduğu için yüksek mimari baskı üretmektedir.

Test Yüzeyi ve Sözleşmeli Testler

Test Alanı	Değer
Frontend test dosyaları	9
Backend test dosyaları	13
Frontend sözleşmeli testler	3
Backend sözleşmeli testler	3
Toplam test LOC	4,096

Table 8: Test yüzeyi ve sözleşmeli test dağılımı.

Bu dağılım klasik kapsam (coverage) oranıyla tam olarak değerlendirilemez. Bazı testler yalnızca satır kapsamı sağlamamakta; aynı zamanda mimari invariantları sabitlemektedir. Özellikle sözleşmeli testler, frontend ve backend arasında elle senkronize edilen davranışları koruduğu için yüksek kaldırma etkisine sahiptir.

Geometri Özeti ve Yeniden Yapılandırma Sırası

Kod tabanı asimetriktir: frontend tarafında Page A çevresinde güçlü bir gravity well oluşmuş, `PageA.types.ts` ve `api/types.ts` iskelet düğümlere dönüşmüş, notes companion dosyaları Page A çevresinde invariant yoğunluğu oluşturmuş, backend tarafı ise görece daha homojen ve katmanlı bir dağılım göstermiştir.

Bu ölçümlerden çıkan yeniden yapılandırma sırası şu biçimde formüle edilebilir:

1. `PageA.types.ts` dosyası, gerçek kullanım alanı dikkate alınarak uygun shared type alanına taşınmalıdır.
2. Sayfalar arası doğrudan içe aktarma ilişkileri temizlenmeli, ortak davranışlar shared hook veya shared component düzeyine indirilmelidir.
3. `api/types.ts` domain bazlı parçalara ayrılmalı, boundary contract tek dosyada şişmiş halde bırakılmamalıdır.
4. Page A içerisindeki hook orkestrasyon yapısı davranış alanlarına göre yeniden ayrıştırılmalıdır.
5. Notes companion dosyaları yeniden yapılandırma sürecinde korunmalı, taşınmalı veya güncellenmelidir.

Sonuç

Bu sıralama, dosya büyüklüğüne göre değil mimari kaldıraç etkisine göre belirlenmiştir. İlk adımlar daha mekanik ve yüksek etkili iken sonraki adımlar daha fazla tasarım kararı gerektirmektedir. Vaka incelemesi, soyut geometrik kavramların (kütle, merkezilik, gravity well) gerçek bir kod tabanında ölçülebilir karşılıklar bulduğunu ve bu ölçümlerin yeniden yapılandırma önceliklendirmesinde doğrudan operasyonel değer taşıdığını göstermektedir.